

Self similar diffusion

September 15, 2026

1 Navier–Stokes

Dimensional incompressible Navier–Stokes equations,

$$\frac{\partial u_i^*}{\partial t^*} + u_j^* \frac{\partial u_i^*}{\partial x_j^*} = -\frac{1}{\rho} \frac{\partial p^*}{\partial x_i^*} + \nu \frac{\partial^2 u_i^*}{\partial x_j^* \partial x_j^*}, \quad (1)$$

together with the continuity equation,

$$\frac{\partial u_i^*}{\partial x_i^*} = 0. \quad (2)$$

We introduce the following nondimensionalization, with velocity and length scales U and L taken to be functions of t :

$$u_i = \frac{u_i^*}{U}, \quad (3)$$

$$x_i = \frac{x_i^*}{L}, \quad (4)$$

$$p = \frac{p^*}{\rho U^2}, \quad (5)$$

$$\frac{dt}{dt^*} = \frac{U^2}{\nu}. \quad (6)$$

We will also need, for each component j ,

$$\frac{dx_j}{dt} = -x_j^* \frac{L'}{L^2} = -x_j \frac{L'}{L}, \quad (7)$$

where prime denotes differentiation with respect to t . Substituting this nondimensionalization and proceeding term by term, the time-derivative term becomes

$$\frac{\partial u_i^*}{\partial t^*} = \left(\frac{dU}{dt} \frac{dt}{dt^*} \right) u_i + U \left(\frac{\partial u_i}{\partial t} \frac{dt}{dt^*} + \frac{\partial u_i}{\partial x_j} \frac{dx_j}{dt} \frac{dt}{dt^*} \right), \quad (8)$$

Substituting (6) and (7), this becomes

$$\frac{\partial u_i^*}{\partial t^*} = U' \frac{U^2}{\nu} u_i + U \left(\frac{\partial u_i}{\partial t} \frac{U^2}{\nu} - \frac{\partial u_i}{\partial x_j} x_j \frac{L'}{L} \frac{U^2}{\nu} \right). \quad (9)$$

The convection term is

$$u_j^* \frac{\partial u_i^*}{\partial x_j^*} = \frac{U^2}{L} u_j \frac{\partial u_i}{\partial x_j}. \quad (10)$$

The pressure term is

$$\frac{1}{\rho} \frac{\partial p^*}{\partial x_i^*} = \frac{U^2}{L} \frac{\partial p}{\partial x_i}. \quad (11)$$

The viscous term is

$$\nu \frac{\partial^2 u_i^*}{\partial x_j^* \partial x_j^*} = \frac{\nu U}{L^2} \frac{\partial^2 u_i}{\partial x_j \partial x_j}. \quad (12)$$

Substituting (9), (10), (11), and (12) into (1) gives

$$\frac{U^2 U'}{\nu} u_i + \frac{U^3}{\nu} \frac{\partial u_i}{\partial t} - \frac{U^3}{\nu} \frac{L'}{L} x_j \frac{\partial u_i}{\partial x_j} + \frac{U^2}{L} u_j \frac{\partial u_i}{\partial x_j} = -\frac{U^2}{L} \frac{\partial p}{\partial x_i} + \frac{\nu U}{L^2} \frac{\partial^2 u_i}{\partial x_j \partial x_j}. \quad (13)$$

Dividing through by U^3/ν and introducing the instantaneous Reynolds number

$$Re = \frac{UL}{\nu}, \quad \text{so that} \quad \frac{\nu}{UL} = \frac{1}{Re}, \quad (14)$$

gives the nondimensional momentum equation

$$\frac{\partial u_i}{\partial t} = \frac{L'}{L} x_j \frac{\partial u_i}{\partial x_j} - \frac{U'}{U} u_i - \frac{1}{Re} u_j \frac{\partial u_i}{\partial x_j} - \frac{1}{Re} \frac{\partial p}{\partial x_i} + \frac{1}{Re^2} \frac{\partial^2 u_i}{\partial x_j \partial x_j}. \quad (15)$$

The dimensional continuity equation is simply

$$\frac{\partial u_i}{\partial x_i} = 0. \quad (16)$$

2 1D Diffusion equation

2.1 Governing equations

The 1D diffusion equation is given by

$$\frac{\partial u^*}{\partial t^*} = D \frac{\partial^2 u^*}{\partial (x^*)^2}. \quad (17)$$

We use the following non-dimensionalization

$$u = \frac{u^*}{U}, \quad (18)$$

$$x = \frac{x^*}{L}, \quad (19)$$

where U and L are functions of time. For the temporal coordinate, we define

$$t^* = s(t), \quad (20)$$

such that

$$\frac{ds}{dt} = s' = \frac{L_k^2}{D}, \quad (21)$$

and

$$\frac{dt}{dt^*} = \frac{D}{L_k^2}. \quad (22)$$

Using the chain rule, the left-hand side of (17) becomes

$$\frac{\partial(uU)}{\partial t^*} = \frac{\partial(uU)}{\partial t} \frac{dt}{dt^*} + U \frac{\partial u}{\partial x} \frac{\partial x}{\partial t} \frac{dt}{dt^*}. \quad (23)$$

Differentiating (19) with respect to t at fixed x^* yields

$$\frac{dx}{dt} = -\frac{x^* L_k'}{L_k^2} = -\frac{L_k'}{L_k} x. \quad (24)$$

Substituting (24) and (22) into (23) gives

$$\begin{aligned}\frac{\partial(uU)}{\partial t^*} &= \left(U \frac{\partial u}{\partial t} + uU' - U \frac{L'_k}{L_k} x \frac{\partial u}{\partial x} \right) \frac{D}{L_k^2} \\ &= U \left(\frac{\partial u}{\partial t} + u \frac{U'}{U} - \frac{L'_k}{L_k} x \frac{\partial u}{\partial x} \right) \frac{D}{L_k^2}.\end{aligned}\quad (25)$$

We now transform the right-hand side of (17):

$$D \frac{\partial^2(uU)}{\partial (x^*)^2} = DU \frac{\partial^2 u}{\partial (x^*)^2} = DU \frac{\partial}{\partial x^*} \left(\frac{\partial u}{\partial x} \frac{\partial x}{\partial x^*} \right). \quad (26)$$

Substituting the spatial derivative scale

$$\frac{dx}{dx^*} = \frac{1}{L_k}, \quad (27)$$

and simplifying, we obtain

$$D \frac{\partial^2(uU)}{\partial (x^*)^2} = DU \frac{\partial}{\partial x} \left(\frac{\partial u}{\partial x} \frac{1}{L_k} \right) \frac{1}{L_k} = \frac{DU}{L_k^2} \frac{\partial^2 u}{\partial x^2}. \quad (28)$$

Equating (25) and (28) and dividing by common factors, we arrive at the transformed equation:

$$\frac{\partial u}{\partial t} = \frac{L'_k}{L_k} x \frac{\partial u}{\partial x} - \frac{U'}{U} u + \frac{\partial^2 u}{\partial x^2}. \quad (29)$$

We assume that U is of the form

$$U = (t^* + \alpha_U)^{\beta_U}, \quad L = (t^* + \alpha_L)^{\beta_L}. \quad (30)$$

It follows that

$$\frac{U_{k+i}}{U_k} = \left(1 + \frac{i \cdot \Delta t_k^*}{t_k^* + \alpha_U} \right)^{\beta_U}, \quad \frac{L_{k+i}}{L_k} = \left(1 + \frac{i \cdot \Delta t_k^*}{t_k^* + \alpha_L} \right)^{\beta_L}. \quad (31)$$

We can express Δt^* using

$$\Delta t_k^* = \int_{t_k}^{t_{k+1}} \frac{L^2}{D} dt \approx \frac{\Delta t}{12D} (23L_k^2 - 16L_{k-1}^2 + 5L_{k-2}^2). \quad (32)$$

As such

$$\frac{U_{k+i}}{U_k} = \left(1 + \frac{i \cdot \Delta t}{12D} \frac{23L_k^2 - 16L_{k-1}^2 + 5L_{k-2}^2}{U_k^{1/\beta_U}} \right)^{\beta_U}, \quad \frac{L_{k+i}}{L_k} = \left(1 + \frac{i \cdot \Delta t}{12D} \frac{23L_k^2 - 16L_{k-1}^2 + 5L_{k-2}^2}{L_k^{1/\beta_L}} \right)^{\beta_L}. \quad (33)$$

We store $\ln(U_k)$ and $\ln(L_k)$ instead of U_k and L_k to avoid overflow. We also perform computations in the logarithmic number system where multiplication and division become addition and subtraction and addition and subtraction are given by

$$\log_b(X + Y) = \log_b(X) + \log_b(1 + b^{y-x}), \quad \log_b(X - Y) = \log_b(X) + \log_b(1 - b^{y-x}) \quad (X > Y > 0), \quad (34)$$

where $x = \log_b(X)$, $y = \log_b(Y)$.

2.2 Integration

The numerical scheme will use different operator splitting schemes which are composed of the diffusion step

$$\Phi_{\Delta t}^{(D)}(u_k) = u_k + \int_{t_k}^{t_k + \Delta t} \frac{\partial^2 u}{\partial x^2} dt, \quad (35)$$

and the scaling step

$$\Phi_{\Delta t}^{(S)}(u_k) = u_k + \int_{t_k}^{t_{k+1}} \left(\frac{L'}{L} x \frac{\partial u}{\partial x} - \frac{U'}{U} u \right) dt. \quad (36)$$

2.2.1 scaling operator

evaluating $\Phi_{\Delta t}^{(S)}$ can be done by solving the following pde

$$\frac{\partial u}{\partial t} = x \frac{L'}{L} \frac{\partial u}{\partial x} - \frac{U'}{U} u, \quad (37)$$

with the method of characteristics. Along a curve $x(t)$ satisfying

$$\frac{dx}{dt} = -x \frac{L'}{L}, \quad (38)$$

the total derivative of u is

$$\frac{du}{dt} = \frac{\partial u}{\partial t} + \frac{\partial u}{\partial x} \frac{dx}{dt} = \frac{\partial u}{\partial t} - x \frac{L'}{L} \frac{\partial u}{\partial x}, \quad (39)$$

so along this characteristic the PDE reduces to the ODE

$$\frac{du}{dt} = -\frac{U'}{U} u. \quad (40)$$

Integrating (38) from t_k to t_{k+1} gives

$$\ln x_{k+1} - \ln x_k = -[\ln L_{k+1} - \ln L_k] \implies x_{k+1} = x_k \frac{L_k}{L_{k+1}}, \quad (41)$$

so that the characteristic arriving at x_{k+1} originated at

$$x_k = x_{k+1} \frac{L_{k+1}}{L_k}. \quad (42)$$

Integrating (40) in the same way gives

$$u_{k+1} = u_k \frac{U_k}{U_{k+1}}. \quad (43)$$

Combining this with the initial condition $u_k(x)$ and the characteristic origin (42) gives the solution of the PDE at $t_{k+1} = t_k + \Delta t$, i.e. the scaling operator in closed form,

$$\Phi_{\Delta t}^{(S)}(u_k)(x) = u_{k+1}(x) = \frac{U_k}{U_{k+1}} u_k\left(x \frac{L_{k+1}}{L_k}\right). \quad (44)$$

Substituting in

$$\tilde{U}_{k+1} = \frac{U_{k+1}}{U_k}, \quad (45)$$

$$\tilde{L}_{k+1} = \frac{L_{k+1}}{L_k}, \quad (46)$$

we have

$$\Phi_{\Delta t}^{(S)}(u_k)(x) = u_{k+1}(x) = \frac{1}{\tilde{U}_{k+1}} u_k\left(x \tilde{L}_{k+1}\right). \quad (47)$$

2.2.2 Spectral interpolation

From Eq. (47), for a real-valued field u_k , u_{k+1} can be computed using spectral interpolation via the non-negative frequency components obtained from a real Fast Fourier Transform (RFFT):

$$\begin{aligned} u_{k+1}(x) = \frac{1}{\tilde{U}_{k+1}} & \left\{ \hat{u}_0^{(k)} + 2 \sum_{j=1}^{N/2-1} \left[\operatorname{Re}(\hat{u}_j^{(k)}) \cos\left(\frac{2\pi j \tilde{L}_{k+1} x}{H}\right) \right. \right. \\ & \left. \left. - \operatorname{Im}(\hat{u}_j^{(k)}) \sin\left(\frac{2\pi j \tilde{L}_{k+1} x}{H}\right) \right] \right. \\ & \left. + \hat{u}_{N/2}^{(k)} \cos\left(\frac{\pi N \tilde{L}_{k+1} x}{H}\right) \right\}, \end{aligned} \quad (48)$$

where $\hat{u}_j^{(k)} = \frac{1}{N} \text{RFFT}(u_k)_j$ for $j = 0, 1, \dots, N/2$, the Nyquist term at $j = N/2$ is included only if N is even, and H is the fixed physical domain length of §2.3.1.

2.2.3 Moment constraints

$\tilde{U}_{k+1}$ and $\tilde{L}_{k+1}$ are set such that the resulting solution has a specific first order moment, M_0 and second order moment, M_2 . Denote the moments of the input u_k as

$$M_0^* = \int u_k dx, \quad (49)$$

and

$$M_2^* = \int x^2 u_k dx. \quad (50)$$

The requirement for the first moment

$$M_0 = \int \frac{1}{\tilde{U}_{k+1}} u_k(x \tilde{L}_{k+1}) dx = \frac{1}{\tilde{U}_{k+1} \tilde{L}_{k+1}} M_0^*. \quad (51)$$

The second moment requirement is

$$M_2 = \frac{1}{M_0} \int x^2 \frac{1}{\tilde{U}_{k+1}} u_k(x \tilde{L}_{k+1}) dx. \quad (52)$$

Substituting $y = x \tilde{L}_{k+1}$ in the integral gives

$$\int x^2 \frac{1}{\tilde{U}_{k+1}} u_k(x \tilde{L}_{k+1}) dx = \frac{1}{\tilde{U}_{k+1} \tilde{L}_{k+1}^3} \int y^2 u_k(y) dy = \frac{M_2^*}{\tilde{U}_{k+1} \tilde{L}_{k+1}^3}, \quad (53)$$

so that (52) becomes

$$M_2 = \frac{M_2^*}{M_0 \tilde{U}_{k+1} \tilde{L}_{k+1}^3}. \quad (54)$$

Using the first-moment requirement in the form $M_0 \tilde{U}_{k+1} \tilde{L}_{k+1} = M_0^*$ to eliminate $\tilde{U}_{k+1}$, this reduces to

$$M_2 = \frac{M_2^*}{M_0^* \tilde{L}_{k+1}^2}, \quad (55)$$

which gives

$$\tilde{L}_{k+1} = \sqrt{\frac{M_2^*}{M_0^* M_2}}. \quad (56)$$

Substituting (56) back into the first-moment requirement $\tilde{U}_{k+1} \tilde{L}_{k+1} = M_0^*/M_0$ then yields

$$\tilde{U}_{k+1} = \frac{M_0^*}{M_0 \tilde{L}_{k+1}} = \frac{(M_0^*)^{3/2}}{M_0} \sqrt{\frac{M_2}{M_2^*}}. \quad (57)$$

2.3 Algorithm

2.3.1 Domain, grid, and notation

The spatial domain is periodic with a fixed physical length H and N grid points; both are simulation parameters, fixed for the whole run, and are distinct from the time-varying self-similar length scale L_k (hence the different symbol). The domain is centered at the origin, $x \in [-H/2, H/2]$, with uniform spacing $\Delta x = H/N$ and grid points

$$x_n = -\frac{H}{2} + n \Delta x, \quad n = 0, 1, \dots, N-1 \quad (58)$$

(indices are 0-based here, as is conventional for FFTs; shift by one in a 1-indexed language such as Julia). Every integral over x below – in particular the moments M_0, M_2, M_0^*, M_2^* – is approximated by the Riemann sum $\int f(x) dx \approx \Delta x \sum_{n=0}^{N-1} f(x_n)$ on this grid. The field u_k always lives on this same fixed grid: rescaling never changes the grid itself, it resamples the field onto it via the spectral interpolation of Eq. (47) and the equation that follows it.

2.3.2 Initialization

A run is specified by the initial field $u_0(x_n)$, the diffusivity D , the domain parameters H, N , a fixed nondimensional timestep Δt , a time horizon T_{end} , the splitting scheme $scheme \in \{\text{LIETROTTER}, \text{STRANG}\}$ (§2.3.5, §2.3.6), and initial guesses $\beta_U^{(0)}, \beta_L^{(0)}$, which only seed the bracket of the first root-find for β_U, β_L (§2.3.7). From these:

- The number of steps is $N_{\text{steps}} = \lceil T_{\text{end}}/\Delta t \rceil$.
- The self-similar frame is initialized at the initial condition itself, $L_0 = U_0 = 1$, i.e. $\ln L_0 = \ln U_0 = 0$.
- The *target* moments M_0, M_2 used in the moment constraints of §2.2.3 are fixed once, from u_0 itself,

$$M_0 = \Delta x \sum_n u_0(x_n), \quad M_2 = \Delta x \sum_n x_n^2 u_0(x_n), \quad (59)$$

and are never recomputed: all subsequent growth of mass and spread is absorbed into U_k, L_k rather than changing M_0, M_2 .

- The flag β_{ok} , which records whether the current β_U, β_L are usable by STRANG, starts as $\beta_{\text{ok}} = \text{false}$.

2.3.3 Top-level integrator

Algorithm 1 Top-level integrator

Require: $u_0, D, H, N, \Delta t, T_{\text{end}}, scheme, \beta_U^{(0)}, \beta_L^{(0)}$

- 1: $N_{\text{steps}} \leftarrow \lceil T_{\text{end}}/\Delta t \rceil$
- 2: $\ln U_0 \leftarrow 0, \ln L_0 \leftarrow 0$
- 3: Compute target moments M_0, M_2 from u_0 via (59)
- 4: $\beta_U \leftarrow \beta_U^{(0)}, \beta_L \leftarrow \beta_L^{(0)}, \beta_{\text{ok}} \leftarrow \text{false}$
- 5: Store $(u_0, \ln U_0, \ln L_0, \beta_U, \beta_L, \beta_{\text{ok}})$
- 6: **for** $k = 0, 1, \dots, N_{\text{steps}} - 1$ **do**
- 7: **if** $scheme = \text{STRANG}$ **and** β_{ok} **then**
- 8: $(u_{k+1}, \ln U_{k+1}, \ln L_{k+1}) \leftarrow \text{STRANG}(u_k, \ln U_k, \ln L_{k-2:k}, \beta_U, \beta_L)$ ▷ §2.3.6
- 9: **else**
- 10: $(u_{k+1}, \ln U_{k+1}, \ln L_{k+1}) \leftarrow \text{LIETROTTER}(u_k, \ln U_k, \ln L_k)$ ▷ §2.3.5
- 11: **end if**
- 12: $u_{k+1}(x_0) \leftarrow 0; u_{k+1}(x_{N-1}) \leftarrow 0$ ▷ zero the two edge grid points
- 13: **if** $k + 1 \geq 3$ **then**
- 14: $(\beta_U, \beta_L, \beta_{\text{ok}}) \leftarrow \text{ESTIMATEBETA}(\ln U_{k:k+1}, \ln L_{k-2:k+1}, \beta_U, \beta_L)$ ▷ §2.3.7
- 15: **end if**
- 16: Store $(u_{k+1}, \ln U_{k+1}, \ln L_{k+1}, \beta_U, \beta_L, \beta_{\text{ok}})$
- 17: **end for**

Throughout, all arithmetic on U_k, L_k (which can be extremely large or small) is done in log space – adding and subtracting logarithms rather than multiplying and dividing the values themselves – with exponentiation deferred until a final value is actually needed (e.g. for output), to avoid overflow or underflow. An index range abbreviates the corresponding tuple of stored values, e.g. $\ln L_{k-2:k} = (\ln L_{k-2}, \ln L_{k-1}, \ln L_k)$. The run constants $D, \Delta t, H, N, M_0, M_2$ are visible to every subroutine and are omitted from their signatures.

The run parameter *scheme* selects the step function. STRANG needs usable estimates of β_U, β_L , so the loop falls back to LIETROTTER whenever β_{ok} is false. This is always the case for the first three steps $k = 0, 1, 2$: the first β update needs four stored states, so it first runs at the end of step $k = 2$. It is also the case for every step that follows a failed β update (§2.3.7), until a later update succeeds. The β update itself runs regardless of *scheme*; under *scheme* = LIETROTTER the estimates are diagnostic only.

2.3.4 Substeps

Both splitting schemes are assembled from the four substeps below. None of them touches the edge grid points, which are handled only by the top-level loop.

Diffuse($u, \Delta t$). Because the diffusion term in (29) has unit coefficient in the nondimensional time t , $\Phi_{\Delta t}^{(D)}$ (Eq. (35)) has the closed-form spectral solution

$$\hat{u}_j^{(D)} = \hat{u}_j e^{-k_j^2 \Delta t}, \quad k_j = \frac{2\pi j}{H}, \quad j = 0, \dots, \lfloor N/2 \rfloor, \quad (60)$$

where $\hat{u}_j = \text{RFFT}(u)_j$: this is an exact one-shot spectral propagator, not an approximate time-marching scheme, so no sub-stepping or implicit solve is needed. Inverse-transforming gives the diffused field $u^{(D)}(x_n)$.

Scale($u, \ln \tilde{U}, \ln \tilde{L}$). Applies the scaling operator (47) with prescribed ratios $\tilde{U}, \tilde{L}$, returning $\tilde{U}^{-1}u(x_n \tilde{L})$ at each grid point x_n . It is evaluated with the spectral interpolation formula following Eq. (47), using the RFFT coefficients of u . Here, and wherever the substeps are used, $\tilde{U}$ and $\tilde{L}$ denote the ratio of U , respectively L , after the substep to its value before it. Inside a Lie–Trotter step this is $\tilde{U}_{k+1} = U_{k+1}/U_k$ of §2.2.3; inside a Strang step it is a ratio across a half step, $U_{k+1/2}/U_k$ or $U_{k+1}/U_{k+1/2}$ (§2.3.6).

ConstrainedScale($u, \ln U, \ln L$). Rescales u so that the result has the fixed target moments M_0, M_2 of (59), and advances the log-scale state accordingly:

1. **Moments of the input**,

$$M_0^* = \Delta x \sum_n u(x_n), \quad M_2^* = \Delta x \sum_n x_n^2 u(x_n). \quad (61)$$

2. **Ratios from the moment constraints** (§2.2.3),

$$\tilde{L} = \sqrt{\frac{M_2^*}{M_0^* M_2}}, \quad \tilde{U} = \frac{(M_0^*)^{3/2}}{M_0} \sqrt{\frac{M_2}{M_2^*}}. \quad (62)$$

3. **Rescale**, $u' = \text{SCALE}(u, \ln \tilde{U}, \ln \tilde{L})$.

4. **Return** u' together with the advanced state $\ln U + \ln \tilde{U}, \ln L + \ln \tilde{L}$.

No time increment enters **CONSTRAINEDSCALE**: the ratios are fixed by the moments of the field it is given, not by the elapsed time. The same substep therefore serves as the full scaling step of Lie–Trotter and as the closing scaling half step of Strang.

PredictScale($i, \ln U_k, \ln L_{k-2:k}, \beta_U, \beta_L$). Predicts the ratios over the fraction i of step k from the power-law form (30), rather than from moments. Writing (32) in log space,

$$\ln \Delta t_k^* = \ln \frac{\Delta t}{12D} + \ln(23 e^{2 \ln L_k} - 16 e^{2 \ln L_{k-1}} + 5 e^{2 \ln L_{k-2}}), \quad (63)$$

with the sum evaluated via (34), and taking the logarithm of (33) gives, for $X \in \{U, L\}$,

$$\ln \frac{X_{k+i}}{X_k} = G_X(i, \beta_X; k), \quad G_X(i, \beta; k) = \beta \ln \left(1 + \exp \left(\ln i + \ln \Delta t_k^* - \frac{\ln X_k}{\beta} \right) \right). \quad (64)$$

PREDICTSCALE returns $(G_U(i, \beta_U; k), G_L(i, \beta_L; k))$. The same function G_X is inverted for β in §2.3.7.

2.3.5 LieTrotter: Lie–Trotter splitting

A Lie–Trotter step composes a full diffusion step with a full scaling step, $u_{k+1} = \Phi_{\Delta t}^{(S)}(\Phi_{\Delta t}^{(D)}(u_k))$, with the scaling ratios fixed by the moment constraints. It uses no estimate of β_U, β_L , so it is valid from the first step. It is written for a general input state $(u, \ln U, \ln L)$ rather than only a stored state $(u_k, \ln U_k, \ln L_k)$, so that STRANG can call it on a half-stepped state.

Algorithm 2 Lie–Trotter step

```

1: function LIETROTTER( $u, \ln U, \ln L$ )
2:    $u^{(D)} \leftarrow \text{DIFFUSE}(u, \Delta t)$  ▷ (60)
3:   return CONSTRAINEDSCALE( $u^{(D)}, \ln U, \ln L$ ) ▷ §2.3.4
4: end function

```

2.3.6 Strang: Strang splitting

A Strang step is the symmetric composition

$$u_{k+1} = \Phi_{\Delta t/2}^{(S)}\left(\Phi_{\Delta t}^{(D)}\left(\Phi_{\Delta t/2}^{(S)}(u_k)\right)\right) \quad (65)$$

of a scaling half step, a full diffusion step, and a second scaling half step. The last two of these are exactly a Lie–Trotter step – a full diffusion step followed by a scaling step whose ratios come from the moment constraints – so STRANG is a predicted scaling half step followed by a call to LIETROTTER.

Algorithm 3 Strang step

```

1: function STRANG( $u_k, \ln U_k, \ln L_{k-2:k}, \beta_U, \beta_L$ )
2:    $(\ln(U_{k+1/2}/U_k), \ln(L_{k+1/2}/L_k)) \leftarrow \text{PREDICTSCALE}(\frac{1}{2}, \ln U_k, \ln L_{k-2:k}, \beta_U, \beta_L)$  ▷ (66)
3:    $u_{k+1/2} \leftarrow \text{SCALE}(u_k, \ln(U_{k+1/2}/U_k), \ln(L_{k+1/2}/L_k))$ 
4:    $\ln U_{k+1/2} \leftarrow \ln U_k + \ln(U_{k+1/2}/U_k); \ln L_{k+1/2} \leftarrow \ln L_k + \ln(L_{k+1/2}/L_k)$ 
5:   return LIETROTTER( $u_{k+1/2}, \ln U_{k+1/2}, \ln L_{k+1/2}$ ) ▷ §2.3.5
6: end function

```

No moments constrain the state at $t_k + \Delta t/2$, so the first scaling half step takes its ratios from the current power-law estimates, via PREDICTSCALE with $i = 1/2$:

$$\ln \frac{U_{k+1/2}}{U_k} = G_U\left(\frac{1}{2}, \beta_U; k\right), \quad \ln \frac{L_{k+1/2}}{L_k} = G_L\left(\frac{1}{2}, \beta_L; k\right). \quad (66)$$

The call to LIETROTTER then diffuses $u_{k+1/2}$ over the full Δt , and its CONSTRAINEDSCALE substep – the second scaling half step – sets $U_{k+1}/U_{k+1/2}$ and $L_{k+1}/L_{k+1/2}$ so that u_{k+1} satisfies the moment constraints. The half-step state $(u_{k+1/2}, \ln U_{k+1/2}, \ln L_{k+1/2})$ is internal to the step and is not stored. The top-level loop calls STRANG only when β_{ok} is true (§2.3.3), which guarantees that β_U, β_L were fitted to the stored history at the end of the previous step and that $\ln L_{k-2}$ exists.

2.3.7 EstimateBeta: estimating β_U and β_L

After every step with $k+1 \geq 3$, regardless of *scheme*, β_U and β_L are re-estimated from the log-scale history. Four stored states are needed: $\ln X_k$ and $\ln X_{k+1}$ for the ratio, and $\ln L_{k-2:k}$ for Δt_k^* . The update inverts the function G_X of (64) with $i = 1$, the same function that PREDICTSCALE evaluates forward with $i = 1/2$:

$$G_U(1, \beta_U; k) = \ln U_{k+1} - \ln U_k, \quad (67)$$

and identically with $U \rightarrow L$ for β_L . Each of (67) and its L -counterpart is a single scalar equation in a single unknown (β_U , resp. β_L), solved by **brentq**; the implementer will need to choose a bracketing interval (e.g. centered on the previous β_U, β_L estimate – on $\beta_U^{(0)}, \beta_L^{(0)}$ for the first update – widened adaptively if it fails to bracket a root).

If both root-finds succeed, ESTIMATEBETA returns the new β_U, β_L with $\beta_{\text{ok}} = \text{true}$. If either fails to bracket a root even after widening, the failed estimate keeps its previous value (which seeds the next bracket), and ESTIMATEBETA returns $\beta_{\text{ok}} = \text{false}$, so the next step falls back to LIETROTTER. Before $k + 1 \geq 3$ no update is made: β_U, β_L stay at $\beta_U^{(0)}, \beta_L^{(0)}$ and β_{ok} stays false.

3 Post-processing

Assume that L and U are of the form

$$L = (t^* + a_L)^{b_L}, \quad (68)$$

and

$$U = (t^* + a_U)^{b_U}. \quad (69)$$

Define

$$\mathcal{L}_k = \ln L_k, \quad \mathcal{U}_k = \ln U_k, \quad (70)$$

together with the per-step log-ratios

$$\ell_k = \ln L_{k+1} - \ln L_k, \quad \lambda_k = \ln U_{k+1} - \ln U_k, \quad (71)$$

which for a Lie–Trotter step equal $\ln \tilde{L}_{k+1}$ and $\ln \tilde{U}_{k+1}$, and for a Strang step combine both half-step ratios.

$$\mathcal{L}_k = \mathcal{L}_0 + \sum_{i=1}^k \ell_i = \ln(L_k), \quad \mathcal{U}_k = \mathcal{U}_0 + \sum_{i=1}^k \lambda_i = \ln(U_k). \quad (72)$$

grows only linearly in k ; unlike L_k and U_k themselves, $\mathcal{L}_k$ and $\mathcal{U}_k$ never overflow.

Differentiating (68) and (69) with respect to t^* gives

$$t^* + a_L = L^{1/b_L}, \quad t^* + a_U = U^{1/b_U}. \quad (73)$$

By the chain rule

$$\alpha = \frac{\frac{dL}{dt^*} \frac{dt^*}{dt}}{L} = \frac{b_L}{t^* + a_L} \frac{L^2}{D}. \quad (74)$$

Substituting (73)

$$\alpha = \frac{b_L}{D} L^{2-1/b_L}. \quad (75)$$

The same steps applied to β give

$$\beta = \frac{b_U}{D} L^2 U^{-1/b_U}. \quad (76)$$

Taking logarithms

$$\ln \alpha_k = \ln \frac{b_L}{D} + \left(2 - \frac{1}{b_L}\right) \mathcal{L}_k, \quad (77)$$

$$\ln \beta_k = \ln \frac{b_U}{D} + 2\mathcal{L}_k - \frac{\mathcal{U}_k}{b_U}. \quad (78)$$

Finally, α_k and β_k are estimated directly from the log-ratios already in hand, without any finite difference of L_k or U_k themselves,

$$\alpha_k \approx \frac{\ell_k}{\Delta t}, \quad \beta_k \approx \frac{\lambda_k}{\Delta t}. \quad (79)$$

By fitting curves to Eq. (77) and Eq. (78) we can compute a_L, a_U, b_L , and b_U .